ITAI 1371

Module 13 Lab Submission

Building Machine Learning Pipelines

Submitted by: Alexander DeBusk

Individual submission begins on the following page.

Module 13 Lab - Building ML Pipelines**Objective:** To understand the importance of `scikit-learn` **Pipelines** for creating robust, reproducible, and professional machine learning workflows.**In this lab, you will refactor code from a previous lab into a clean, professional `Pipeline` object.**

Part 1: Why Use Pipelines?**Concept:** As you've seen, a typical ML workflow involves multiple steps: loading data, cleaning it, splitting it, preprocessing features (scaling, encoding), and finally, training a model. Managing all these steps separately can be messy and error-prone.**Data Leakage:** A major risk of manual preprocessing is **data leakage**. This happens when information from the test set accidentally "leaks" into the training process. For example, if you calculate the mean for scaling using the *entire* dataset before splitting, the model has already "seen" the test data, leading to overly optimistic performance estimates.**A `scikit-learn` Pipeline solves these problems by:**1. **Encapsulating** all workflow steps into a single object.2. **Preventing Data Leakage:** It ensures that preprocessing steps are fitted *only* on the training data during cross-validation or when calling `.fit()`.3. **Improving Reproducibility:** The entire workflow is saved as one object, making it easy to reuse and deploy.

as one object, making it easy to reuse and deploy.

Part 2: The "Manual" Way (What We Did Before)Let's revisit the Titanic dataset and the steps we took to prepare the data and train a model. This code should look familiar. Notice how many separate objects and steps there are.

```
import pandas as pd
import numpy as np

from sklearn.model_selection import train_test_split
from sklearn.impute import SimpleImputer
from sklearn.preprocessing import StandardScaler, OneHotEncoder
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import accuracy_score

# Load data
df = pd.read_csv('https://raw.githubusercontent.com/datasciencedojo/datasets

# Basic feature engineering and cleaning
df['Age'].fillna(df['Age'].median(), inplace=True)
df['Embarked'].fillna(df['Embarked'].mode()[0], inplace=True)
df.drop('Cabin', axis=1, inplace=True)

X = df.drop(['Survived', 'Name', 'Ticket', 'PassengerId'], axis=1)
y = df['Survived']

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

# Identify feature types
numeric_features = ['Age', 'Fare', 'SibSp', 'Parch']
categorical_features = ['Pclass', 'Sex', 'Embarked']

# Manual Preprocessing
scaler = StandardScaler()
X_train_scaled_num = scaler.fit_transform(X_train[numeric_features])
X_test_scaled_num = scaler.transform(X_test[numeric_features])  # Note: usin

encoder = OneHotEncoder(handle_unknown='ignore')
X_train_encoded_cat = encoder.fit_transform(X_train[categorical_features])
X_test_encoded_cat = encoder.transform(X_test[categorical_features])
```

```
# Combine preprocessed features
X_train_processed = np.hstack((
    X_train_scaled_num,
    X_train_encoded_cat.toarray()
))
X_test_processed = np.hstack((
    X_test_scaled_num,
    X_test_encoded_cat.toarray()
))

# Train model
model = RandomForestClassifier(random_state=42)
model.fit(X_train_processed, y_train)

y_pred = model.predict(X_test_processed)

print(f"Accuracy (Manual Method): {accuracy_score(y_test, y_pred):.2%}")
```

```
/tmp/ipykernel_792/4252383106.py:14: FutureWarning: A value is trying to be s
The behavior will change in pandas 3.0. This inplace method will never work b

For example, when doing 'df[col].method(value, inplace=True)', try using 'df.


  df['Age'].fillna(df['Age'].median(), inplace=True)
/tmp/ipykernel_792/4252383106.py:15: FutureWarning: A value is trying to be s
The behavior will change in pandas 3.0. This inplace method will never work b

For example, when doing 'df[col].method(value, inplace=True)', try using 'df.


  df['Embarked'].fillna(df['Embarked'].mode()[0], inplace=True)
Accuracy (Manual Method): 82.68%
```

Part 3: The "Pipeline" WayNow, let's do the exact same thing but encapsulate all the preprocessing steps into a single `Pipeline`.**Your Task:** Use `make_pipeline` and `make_column_transformer` to build a complete workflow. This is the modern, professional way to build models in `scikit-learn`.

```
from sklearn.pipeline import make_pipeline
from sklearn.compose import make_column_transformer
```

```
# Reload the data to start fresh
df = pd.read_csv('https://raw.githubusercontent.com/datasciencedojo/datasets

df.drop(['Cabin', 'Name', 'Ticket', 'PassengerId'], axis=1, inplace=True)

X = df.drop('Survived', axis=1)
y = df['Survived']

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

# --- ENTER YOUR CODE HERE ---

# 1. Create a pipeline for numeric features
#    This pipeline will first impute missing 'Age' values with the median, t
# numeric_transformer = make_pipeline(
#     SimpleImputer(strategy='median'),
#     StandardScaler()
# )

# 2. Create a pipeline for categorical features
#    This pipeline will first impute missing 'Embarked' values with the most
# categorical_transformer = make_pipeline(
#     SimpleImputer(strategy='most_frequent'),
#     OneHotEncoder(handle_unknown='ignore')
# )

# 3. Use ColumnTransformer to apply different transformers to different colu
# preprocessor = make_column_transformer(
#     (numeric_transformer, ['Age', 'Fare', 'SibSp', 'Parch']),
#     (categorical_transformer, ['Pclass', 'Sex', 'Embarked'])
# )

# 4. Create the final, full pipeline
#    This chains the preprocessor and the final model together.
# final_pipeline = make_pipeline(
#     preprocessor,
#     RandomForestClassifier(random_state=42)
# )

# 5. Fit and evaluate the entire pipeline in one step!
# final_pipeline.fit(X_train, y_train)
# y_pred_pipeline = final_pipeline.predict(X_test)
# print(f"Accuracy (Pipeline Method): {accuracy_score(y_test, y_pred_pipelin
```

📝 Reflective Knowledge Check**Instructions:** Answer the

following questions in this markdown cell.1. **Code Comparison:** Look at the "Manual Way" versus the "Pipeline Way". What are the three biggest advantages you see in using the Pipeline approach?2. **Data Leakage Explained:** In the manual code, we used `scaler.fit_transform()` on the training data but only `scaler.transform()` on the test data. Why was this distinction crucial? How does the Pipeline automatically handle this for you?3. **Extending the Pipeline:** Imagine you wanted to add a PCA step to reduce dimensionality *after* scaling and encoding but *before* the RandomForestClassifier. How would you modify your `final_pipeline` object to include this step? (You don't need to write the full code, just describe where you would add `PCA()`.)4. **Real-World Value:** You are handing your model over to another team to deploy into a web application. Why is giving them the single `final_pipeline` object much safer and more reliable than giving them the 5 separate objects (`scaler`, `encoder`, `model`, etc.) from the manual approach?**

1. Code Comparison:

The three biggest advantages of using a Pipeline are that it keeps the code organized, helps prevent data leakage, and makes the workflow easier to reuse. Instead of managing multiple preprocessing and model objects separately, everything is combined into one object. This makes the code cleaner and easier to maintain.

2. Data Leakage Explained:

Using fit_transform() on the training data allows the scaler to learn the mean and

standard deviation from only the training data. Using transform() on the test data applies those same values without learning anything new from the test data. This is important because the model should not learn information from data it hasn't seen during training. The Pipeline handles this automatically by fitting preprocessing steps on the training data and applying them to the test data.

3. Extending the Pipeline:

I would add PCA() between the preprocessor and the RandomForestClassifier inside final_pipeline. This would allow PCA to reduce the number of features after they have been scaled and encoded but before the model is trained.

4. Real-World Value:

Giving another team the single final_pipeline object is safer because it contains all the preprocessing steps and the trained model together. This reduces the chance of missing a step or applying preprocessing incorrectly. It also makes the workflow easier to reproduce and deploy because the other team only needs to use one object instead of managing several separate objects.**

ITAI 1371

Module 13 Lab Submission

Building Machine Learning Pipelines

Submitted by: Shareefah Adeola

Individual submission begins on the following page.

Module 13 Lab - Building ML Pipelines**Objective:** To understand the importance of `scikit-learn` **Pipelines** for creating robust, reproducible, and professional machine learning workflows.**In this lab, you will refactor code from a previous lab into a clean, professional `Pipeline` object.**

Part 1: Why Use Pipelines?**Concept:** As you've seen, a typical ML workflow involves multiple steps: loading data, cleaning it, splitting it, preprocessing features (scaling, encoding), and finally, training a model. Managing all these steps separately can be messy and error-prone.**Data Leakage:** A major risk of manual preprocessing is **data leakage**. This happens when information from the test set accidentally "leaks" into the training process. For example, if you calculate the mean for scaling using the *entire* dataset before splitting, the model has already "seen" the test data, leading to overly optimistic performance estimates.**A `scikit-learn` Pipeline solves these problems by:**1. **Encapsulating** all workflow steps into a single object.2. **Preventing Data Leakage:** It ensures that preprocessing steps are fitted *only* on the training data during cross-validation or when calling `.fit()`.3. **Improving Reproducibility:** The entire workflow is saved as one object, making it easy to reuse and deploy.

Part 2: The "Manual" Way (What We Did Before)Let's revisit the Titanic dataset and the steps we took to prepare the data and train a model. This code should look familiar. Notice how many separate objects and steps there are.

```
# 1. Import necessary libraries
import pandas as pd
import numpy as np

from sklearn.model_selection import train_test_split
from sklearn.impute import SimpleImputer
from sklearn.preprocessing import StandardScaler, OneHotEncoder
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import accuracy_score

# 2. Load the Titanic dataset
df = pd.read_csv(
    'https://raw.githubusercontent.com/datasciencedojo/datasets/master/titanic.csv'
)

# 3. Basic feature engineering and cleaning
df['Age'] = df['Age'].fillna(df['Age'].median())

df['Embarked'] = df['Embarked'].fillna(df['Embarked'].mode()[0])


df.drop('Cabin', axis=1, inplace=True)

# 4. Separate independent and dependent variables
X = df.drop(
    ['Survived', 'Name', 'Ticket', 'PassengerId'],
    axis=1
)

y = df['Survived']

# 5. Split data into training and testing sets
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

# 6. Identify feature types
numeric_features = ['Age', 'Fare', 'SibSp', 'Parch']

categorical_features = ['Pclass', 'Sex', 'Embarked']

# 7. Manual preprocessing - Numerical features
scaler = StandardScaler()

X_train_scaled_num = scaler.fit_transform(
    X_train[numeric_features]
)

X_test_scaled_num = scaler.transform(
    X_test[numeric_features]
)

# 8. Manual preprocessing - Categorical features
encoder = OneHotEncoder(handle_unknown='ignore')

X_train_encoded_cat = encoder.fit_transform(
    X_train[categorical_features]
```

```
)

X_test_encoded_cat = encoder.transform(
    X_test[categorical_features]
)

# 9. Combine preprocessed features
X_train_processed = np.hstack((
    X_train_scaled_num,
    X_train_encoded_cat.toarray()
))

X_test_processed = np.hstack((
    X_test_scaled_num,
    X_test_encoded_cat.toarray()
))

# 10. Train the model
model = RandomForestClassifier(random_state=42)

model.fit(X_train_processed, y_train)

# 11. Make predictions
y_pred = model.predict(X_test_processed)

# 12. Evaluate model accuracy
print(
    f"Accuracy (Manual Method): {accuracy_score(y_test, y_pred):.2%}"
)
```

```
Accuracy (Manual Method): 82.68%
```

Part 3: The "Pipeline" WayNow, let's do the exact same thing but encapsulate all the preprocessing steps into a single `Pipeline`.**Your Task:** Use `make_pipeline` and `make_column_transformer` to build a complete workflow. This is the modern, professional way to build models in `scikit-learn`.

```
# Import Pipeline tools
from sklearn.pipeline import make_pipeline
from sklearn.compose import make_column_transformer

# Reload the data to start fresh
df = pd.read_csv(
    'https://raw.githubusercontent.com/datasciencedojo/datasets/master/titanic.csv'
)

df.drop(
    ['Cabin', 'Name', 'Ticket', 'PassengerId'],
    axis=1,
```

```
    inplace=True
)

X = df.drop('Survived', axis=1)
y = df['Survived']

# Split data into training and testing sets
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

# 1. Create a pipeline for numeric features
numeric_transformer = make_pipeline(
    SimpleImputer(strategy='median'),
    StandardScaler()
)

# 2. Create a pipeline for categorical features
categorical_transformer = make_pipeline(
    SimpleImputer(strategy='most_frequent'),
    OneHotEncoder(handle_unknown='ignore')
)

# 3. Apply different transformers to different columns
preprocessor = make_column_transformer(
    (numeric_transformer, ['Age', 'Fare', 'SibSp', 'Parch']),
    (categorical_transformer, ['Pclass', 'Sex', 'Embarked'])
)

# 4. Create the final, full pipeline
final_pipeline = make_pipeline(
    preprocessor,
    RandomForestClassifier(random_state=42)
)

# 5. Fit and evaluate the entire pipeline
final_pipeline.fit(X_train, y_train)

y_pred_pipeline = final_pipeline.predict(X_test)

print(
    f"Accuracy (Pipeline Method): {accuracy_score(y_test, y_pred_pipeline):.2%}"
)
```

```
Accuracy (Pipeline Method): 82.68%
```

📝 Reflective Knowledge CheckInstructions: Answer the following questions in this markdown cell.

1. Code Comparison: Look at the "Manual Way" versus the "Pipeline Way". What are the three biggest advantages you see in using the Pipeline approach?

Code Comparison: Three Advantages of Using the Pipeline Approach

The three biggest advantages of using the Pipeline approach are:

1. **Simplicity:** A Pipeline combines multiple preprocessing and machine learning steps into one object, making the code easier to read, organize, manage, and maintain.
2. **Prevention of Data Leakage:** A Pipeline ensures that preprocessing steps are fitted only on the training data. This prevents information from the test dataset from influencing the training process.
3. **Reproducibility:** The entire machine learning workflow is stored in one object, making it easier to reuse, reproduce, and deploy the model consistently.

2. **Data Leakage Explained:** In the manual code, we used scaler.fit_transform() on the training data but only scaler.transform() on the test data. Why was this distinction crucial? How does the Pipeline automatically handle this for you?

The distinction between fit_transform() and transform() is crucial because it prevents data leakage.

The fit_transform() function learns the preprocessing parameters, such as the mean and standard deviation, from the training data and then applies the transformation. The transform() function applies those already learned parameters to the test data without learning from it again.

Using fit_transform() on the test data would allow information from the test dataset to influence the training process, resulting in data leakage and potentially unreliable model evaluation.

The Pipeline automatically handles this by fitting the preprocessing steps only on the training data when .fit() is called and applying the learned transformations to the test data when .predict() is called.

3. Extending the Pipeline: Imagine you wanted to add a PCA step to reduce dimensionality after scaling and encoding but before the RandomForestClassifier. How would you modify your final_pipeline object to include this step? (You don't need to write the full code, just describe where you would add PCA().)

To add PCA (Principal Component Analysis) to the Pipeline, I would import PCA from sklearn.decomposition and insert PCA() after the preprocessing step and before the RandomForestClassifier.

The PCA step would reduce the dimensionality of the processed features after scaling and encoding but before the classification model is trained.

This would allow the model to work with a reduced set of features while retaining important information from the dataset.

4. **Real-World Value:** You are handing your model over to another team to deploy into a web application. Why is giving them the single `final_pipeline` object much safer and more reliable than giving them the 5 separate objects (`scaler`, `encoder`, `model`, etc.) from the manual approach?

Giving another team a single final_pipeline object is much safer and more reliable because it combines all the preprocessing steps and the trained machine learning model into one object.

It ensures that the same preprocessing operations are consistently applied to new data before predictions are made. It also reduces the risk of human errors, such as forgetting a preprocessing step or applying transformations incorrectly.

Furthermore, the Pipeline is easier to save, reuse, maintain, and deploy into a web application because the receiving team only needs to work with one complete object instead of managing several separate objects.

ITAI 1371

Module 13 Lab Submission

Building Machine Learning Pipelines

Submitted by: Yilin Leng

Individual submission begins on the following page.

Module 13 Lab - Building ML Pipelines**Objective:** To understand the importance of `scikit-learn` **Pipelines** for creating robust, reproducible, and professional machine learning workflows.**In this lab, you will refactor code from a previous lab into a clean, professional `Pipeline` object.**

Part 1: Why Use Pipelines?**Concept:** As you've seen, a typical ML workflow involves multiple steps: loading data, cleaning it, splitting it, preprocessing features (scaling, encoding), and finally, training a model. Managing all these steps separately can be messy and error-prone.**Data Leakage:** A major risk of manual preprocessing is **data leakage**. This happens when information from the test set accidentally "leaks" into the training process. For example, if you calculate the mean for scaling using the *entire* dataset before splitting, the model has already "seen" the test data, leading to overly optimistic performance estimates.**A `scikit-learn` Pipeline solves these problems by:**1. **Encapsulating** all workflow steps into a single object.2. **Preventing Data Leakage:** It ensures that preprocessing steps are fitted *only* on the training data during cross-validation or when calling `.fit()`.3. **Improving Reproducibility:** The entire workflow is saved as one object, making it easy to reuse and deploy.

Part 2: The "Manual" Way (What We Did Before)Let's revisit the Titanic dataset and the steps we took to prepare the data and train a model. This code should look familiar. Notice how many separate objects and steps there are.

```
import pandas as pd
import numpy as np

from sklearn.model_selection import train_test_split
from sklearn.impute import SimpleImputer
from sklearn.preprocessing import StandardScaler, OneHotEncoder
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import accuracy_score

# Load data
```

```
df = pd.read_csv(
    'https://raw.githubusercontent.com/datasciencedojo/datasets/master/titanic.csv'
)

# Basic feature engineering and cleaning
df['Age'].fillna(df['Age'].median(), inplace=True)
df['Embarked'].fillna(df['Embarked'].mode()[0], inplace=True)

df.drop('Cabin', axis=1, inplace=True)

X = df.drop(['Survived', 'Name', 'Ticket', 'PassengerId'], axis=1)
y = df['Survived']

# Split the data
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

# Identify feature types
numeric_features = ['Age', 'Fare', 'SibSp', 'Parch']
categorical_features = ['Pclass', 'Sex', 'Embarked']

# Manual Preprocessing
scaler = StandardScaler()

X_train_scaled_num = scaler.fit_transform(X_train[numeric_features])
X_test_scaled_num = scaler.transform(X_test[numeric_features])

# Encode categorical features
encoder = OneHotEncoder(handle_unknown='ignore')

X_train_encoded_cat = encoder.fit_transform(X_train[categorical_features])
X_test_encoded_cat = encoder.transform(X_test[categorical_features])

# Combine preprocessed features
X_train_processed = np.hstack((
    X_train_scaled_num,
    X_train_encoded_cat.toarray()
))

X_test_processed = np.hstack((
    X_test_scaled_num,
    X_test_encoded_cat.toarray()
))

# Train model
model = RandomForestClassifier(random_state=42)

model.fit(X_train_processed, y_train)

y_pred = model.predict(X_test_processed)

# Evaluate the model
```

```
print(f"Accuracy (Manual Method): {accuracy_score(y_test, y_pred):.2%}")
```

```
/tmp/ipykernel_1582/4263378427.py:16: FutureWarning: A value is trying to be set on a copy of a DataFrame or Series through chained assignment using an inplace method.
The behavior will change in pandas 3.0. This inplace method will never work because the intermediate object on which we are setting values always behaves as a copy.

For example, when doing 'df[col].method(value, inplace=True)', try using 'df.method({col: value}, inplace=True)' or df[col] = df[col].method(value) instead, to perform the operation inplace on the original object.


  df['Age'].fillna(df['Age'].median(), inplace=True)
/tmp/ipykernel_1582/4263378427.py:17: FutureWarning: A value is trying to be set on a copy of a DataFrame or Series through chained assignment using an inplace method.
The behavior will change in pandas 3.0. This inplace method will never work because the intermediate object on which we are setting values always behaves as a copy.

For example, when doing 'df[col].method(value, inplace=True)', try using 'df.method({col: value}, inplace=True)' or df[col] = df[col].method(value) instead, to perform the operation inplace on the original object.


  df['Embarked'].fillna(df['Embarked'].mode()[0], inplace=True)
Accuracy (Manual Method): 82.68%
```

Part 3: The "Pipeline" WayNow, let's do the exact same thing but encapsulate all the preprocessing steps into a single `Pipeline`.**Your Task:** Use `make_pipeline` and `make_column_transformer` to build a complete workflow. This is the modern, professional way to build models in `scikit-learn`.

```
from sklearn.pipeline import make_pipeline
from sklearn.compose import make_column_transformer

# Reload the data to start fresh
df = pd.read_csv('https://raw.githubusercontent.com/datasciencedojo/datasets/master/titanic.csv')

df.drop(['Cabin', 'Name', 'Ticket', 'PassengerId'], axis=1, inplace=True)

X = df.drop('Survived', axis=1)
y = df['Survived']

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

# 1. Create a pipeline for numeric features
numeric_transformer = make_pipeline(
    SimpleImputer(strategy='median'),
    StandardScaler()
)

# 2. Create a pipeline for categorical features
categorical_transformer = make_pipeline(
    SimpleImputer(strategy='most_frequent'),
    OneHotEncoder(handle_unknown='ignore')
)
```

```
# 3. Use ColumnTransformer to apply different transformers to different columns
preprocessor = make_column_transformer(
    (numeric_transformer, ['Age', 'Fare', 'SibSp', 'Parch']),
    (categorical_transformer, ['Pclass', 'Sex', 'Embarked'])
)

# 4. Create the final, full pipeline
final_pipeline = make_pipeline(
    preprocessor,
    RandomForestClassifier(random_state=42)
)

# 5. Fit and evaluate the entire pipeline
final_pipeline.fit(X_train, y_train)

y_pred_pipeline = final_pipeline.predict(X_test)

print(f"Accuracy (Pipeline Method): {accuracy_score(y_test, y_pred_pipeline):.2%}")
```

```
Accuracy (Pipeline Method): 82.68%
```

📝 Reflective Knowledge Check**Instructions:** Answer the following questions in this markdown cell.1. **Code Comparison:** Look at the "Manual Way" versus the "Pipeline Way". What are the three biggest advantages you see in using the Pipeline approach?2. **Data Leakage Explained:** In the manual code, we used `scaler.fit_transform()` on the training data but only `scaler.transform()` on the test data. Why was this distinction crucial? How does the Pipeline automatically handle this for you?3. **Extending the Pipeline:** Imagine you wanted to add a PCA step to reduce dimensionality *after* scaling and encoding but *before* the RandomForestClassifier. How would you modify your `final_pipeline` object to include this step? (You don't need to write the full code, just describe where you would add `PCA()`.)4. **Real-World Value:** You are handing your model over to another team to deploy into a web application. Why is giving them the single `final_pipeline` object much safer and more reliable than giving them the 5 separate objects (`scaler`, `encoder`, `model`, etc.) from the manual approach?

1. Code Comparison: Look at the "Manual Way" versus the "Pipeline Way". What are the three biggest advantages you see in using the Pipeline approach?

Pipeline makes the code cleaner and easier to manage. It combines preprocessing and model training into one workflow, reduces manual errors, and makes the model easier to reuse.

2. Data Leakage Explained: In the manual code, we used scaler.fit_transform() on the training data but only scaler.transform() on the test data. Why was this distinction crucial? How does the Pipeline automatically handle this for you?

   fit_transform() learns the scaling values from training data, while transform() applies the same values to test data. This prevents test data from influencing the training process. Pipeline handles this automatically.

3. Extending the Pipeline: Imagine you wanted to add a PCA step to reduce dimensionality after scaling and encoding but before the RandomForestClassifier. How would you modify your final_pipeline object to include this step?

   I would add PCA() between the preprocessor and RandomForestClassifier. This allows PCA to reduce the number of features before model training.

4. Real-World Value: Why is giving another team the single final_pipeline object much safer and more reliable than giving them the 5 separate objects (scaler, encoder, model, etc.) from the manual approach?

   A single final_pipeline keeps all preprocessing steps and the model together. This prevents inconsistent data processing and makes the model easier to deploy, maintain, and share with other teams.